UNIVERSITY OF RWANDA

College Of Science and Technology

Computer Engineering

Year *3*

Gako Campus

FLUTTER MOBILE DEVELOPMENT

Reg n°: **223026685**

Names: Jerome HAKIZIMANA

APIS INTEGRATION LAB REPORT

GOAT HUB

CONTENTS

SCANNED DOCUMENT 2
1. INTRODUCTION 6
2. DEPENDENCIES USED AND WHY 7
3. API EXCEPTION HANDLING 7
4. FUTURE WIDGET IMPLEMENTATION 9
5. APPLICATION SCREENSHOTS 10
6. CONCLUSION 17
APPENDIX: SCREENSHOT INDEX 18

TABLE OF FIGURE

Figure 1: pubspec.yaml with all project dependencies 7
Figure 2: Custom error handling UI 8
Figure 4: Successfully loaded consultations displayed as cards &
Figure 3:Loading indicator while waiting for API response 10
Figure 5:Login screen where users enter credentials 10
Figure 6: Registration screen for new users 11
Figure 7: Farmer dashboard showing statistics and quick actions 12
Figure 8: Form for farmers to create a new consultation 13
Figure 9: Farmer viewing their consultations with status indicators 14
Figure 10: Veterinarian viewing all pending consultations 15
Figure 11: Admin view with system-wide statistics 16

SCANNED DOCUMENT

THE FOLLOWING ARE SCANNED HANDWRITTEN DOCUMENT

GOAT HEATH HUB Project

One a piece of paper, discuss the following

Q1. Which dependencies did I use and why?

1. HTTP package (http: ^1.1.2)

I used this to make Api calls to my Django backend. It provides simple methods like get(), post(), put() and delete() that allows my flutter App to communicate with the server. It is lightweight and perfect for REST Api Integration

2. Provider package (provider: ^6.1.1)

This handles state Management in my App. It ensures that when data changes (like when a New Consultation is added), the UI automatically updates to reflect those changes. It separates business logic from UI code, making the App easier to maintain

3. Go Router (go_router: ^13.2.0)

I used this for navigation because my App has three different User Types (farmers, vets and Admins) with different screens. Go Router allows me to control access based on user roles and handles complex navigation patterns easily.

4. Shared preferences (shared_preferences: ^2.2.2)

This stores user login information locally on the device. When a user logs in, I save their details so they don't have to log in again every time they open the App. It's like "remember me" functionality

5. Image picker (image_picker: ^1.0.5)
farmers need to upload pictures of their goats when consulting vets. This package allows user to select images from their gallery or take new photos with camera.

6. Intl (Intl: ^0.20.2)
This helps format dates and times properly when the Api returns timestamps, I use this package to display them in a human-readable format like "March 19, 2026" or "2 hours ago".

Q2. How can you handle Api exceptions (network errors, invalid data)?

* Network Error Handling:
I wrap all Api calls in try-catch block. When there's no internet connection I catch a SocketException and show friendly message: "No internet connection. Please check your network". If the server takes too long to respond, I catch a "TimeoutException" and tell the user: "Connection timeout. Server is not responding."

* HTTP status code handling:
I check the status code of every response:
- 200 (Success) → process the data normally
- 400 (Bad Request) → show validation error from the server.
- 401 (Unauthorized) → Redirect to login page.
- 404 (Not found) → Tell user the resource doesn't exist.
- 500 (Server Error) → shows "Server Error, please try again later"

* Invalid data handling:
I always validate that the data from the Api has the expected format. If required fields are missing I show a fallback value like "Unknown" or "Not provided". This prevent the app from crashing when the Api returns unexpected data.

Q3. Explain the future widget you used and the part of the UI that depends on the Future.

* What is Future?

A future in Dart represent a value that will be available at some point in the future. It is like a promise that data will arrive Eventually. Api calls are perfect Example of future because they take time to Complete.

* How I used Future?

In my Goat Health hub app, I use future for all Api operations:

- Future<User> login() - Returns user data after Authentication.
- Future<list<Consultations>> loadConsultation() - Returns a list of ~~Consulta~~ Consultation.
- Future<bool> create Consultation() - Returns Sucess/fail after creating.

* UI parts that depend on future:

The parts of my ~~app~~ UI that depends on future are Typically data display. for Example:

1. Loading state: When waiting for data, I show a loading spinner with a message like "Loading Consultation..." This tells the user something is happening.

2. Sucess State: When the future Completes Sucessfully, I display the actual data for Consultations, I show cards with farmer names, messages and Status Indicators

3. Error state: if the future Completes with an Error, I show a custom error widget with:
   - An error icon
   - A descriptive message
   - A retry button to Attempt the operation Again.

4. Empty State: if the future returns Empty data, I show a friendly message: "No Consultation yet" with a button to Create one.

1. INTRODUCTION

Goat Health Hub is a mobile application that connects farmers with veterinarians. Farmers can consult vets about their goats' health issues, and veterinarians can provide professional advice. The app communicates with a Django backend API to perform all CRUD operations.

API Endpoints Used:

- POST /api/login/ - User authentication
- POST /api/register/ - New user registration
- GET /api/consultations/ - Fetch all consultations
- POST /api/consultations/create/ - Create new consultation
- POST /api/replies/create/ - Reply to consultation etc..

Some Of example of how My Apis are accessed on login

```
PS D:\MOBILE APP\goathub> flutter run
is in a constructor or an initState() method, then the rebuilt dependent widget will not reflect the changes in the inherited widget.
Typically references to inherited widgets should occur in widget build() methods. Alternatively, initialization based on inherited widgets can be placed in the
didChangeDependencies method, which is called after initState and whenever the dependencies change thereafter.

Performing hot reload...                                        2,134ms
Reloaded application in 2,136ms.
🔵 LoginScreen: Attempting login with:
   Role: farmer
   UserID: Jerome
🔵 AuthProvider: Starting login for role=farmer, userId=Jerome
🔵 AuthApi: Sending login request
   URL: http://localhost:8000/api/login/
   Body: {"role": "farmer", "user_id": "Jerome", "password": "***"}
🔵 AuthApi: Response status: 200
🔵 AuthApi: Response body: {"success": true, "role": "farmer", "user_id": 18, "user_name": "Jerome HAKIZIMANA", "id_number": "Jerome", "phone": "+250786297917", "district":
"Bugesera"}
🔵 AuthProvider: Login response = {success: true, role: farmer, user_id: 18, user_name: Jerome HAKIZIMANA, id_number: Jerome, phone: +250786297917, district: Bugesera}
🔵 AuthProvider: Creating user with type=farmer, id=18, name=Jerome HAKIZIMANA
🔵 AuthProvider: Login successful for user Jerome HAKIZIMANA
🔵 LoginScreen: Login successful for user: Jerome HAKIZIMANA
Could not find a set of Noto fonts to display all missing characters. Please add a font asset for the missing characters. See: https://docs.flutter.dev/cookbook/design/fonts
```

EXAMPLE OF MY APIS ENDPOINTS USED

```dart
class ApiEndpoints {
  static const String baseUrl = AppConstants.baseUrl;
  // Auth endpoints
  static const String login = '/api/login/';
  static const String register = '/api/register/';
  static const String logout = '/api/logout/';
  static const String refreshToken = '/api/token/refresh/';

  // Farmer endpoints
  static const String farmers = '/api/farmers/';
  static String farmerDetail(int id) => '/api/farmers/$id/';
  static String farmerStats(int id) => '/api/farmers/$id/stats/';
  static String farmerConsultations(String name) => '/api/farmers/$name/consultations/';
```

2. DEPENDENCIES USED AND WHY

```yaml
  # Navigation
  go_router: ^17.1.0

  # State Management
  provider: ^6.1.1

  # Networking
  http: ^1.1.2
  dio: ^5.4.0

  # Storage
  shared_preferences: ^2.2.2
  flutter_secure_storage: ^10.0.0

  # Utilities
  intl: ^0.20.2  # For date formatting (updated to match form_builder_validators requirement)
  path_provider: ^2.1.1
  url_launcher: ^6.2.4

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^6.0.0

flutter:
  uses-material-design: true

  assets:
    - assets/images/
    - assets/icons/
```

Figure 1: pubspec.yaml with all project dependencies

3. API EXCEPTION HANDLING

3.1 Types of Exceptions Handled

Exception Type	Cause	User Message
SocketException	No internet connection	"No internet connection. Please check your network."
TimeoutException	Server not responding	"Connection timeout. Server is not responding."
FormatException	Invalid JSON response	"Data format error. Please try again."
400 Bad Request	Invalid user input	Shows specific validation errors
401 Unauthorized	Invalid credentials	"Invalid username or password"
404 Not Found	Resource missing	"The requested information was not found"
500 Server Error	Backend issues	"Server error. Please try again later."

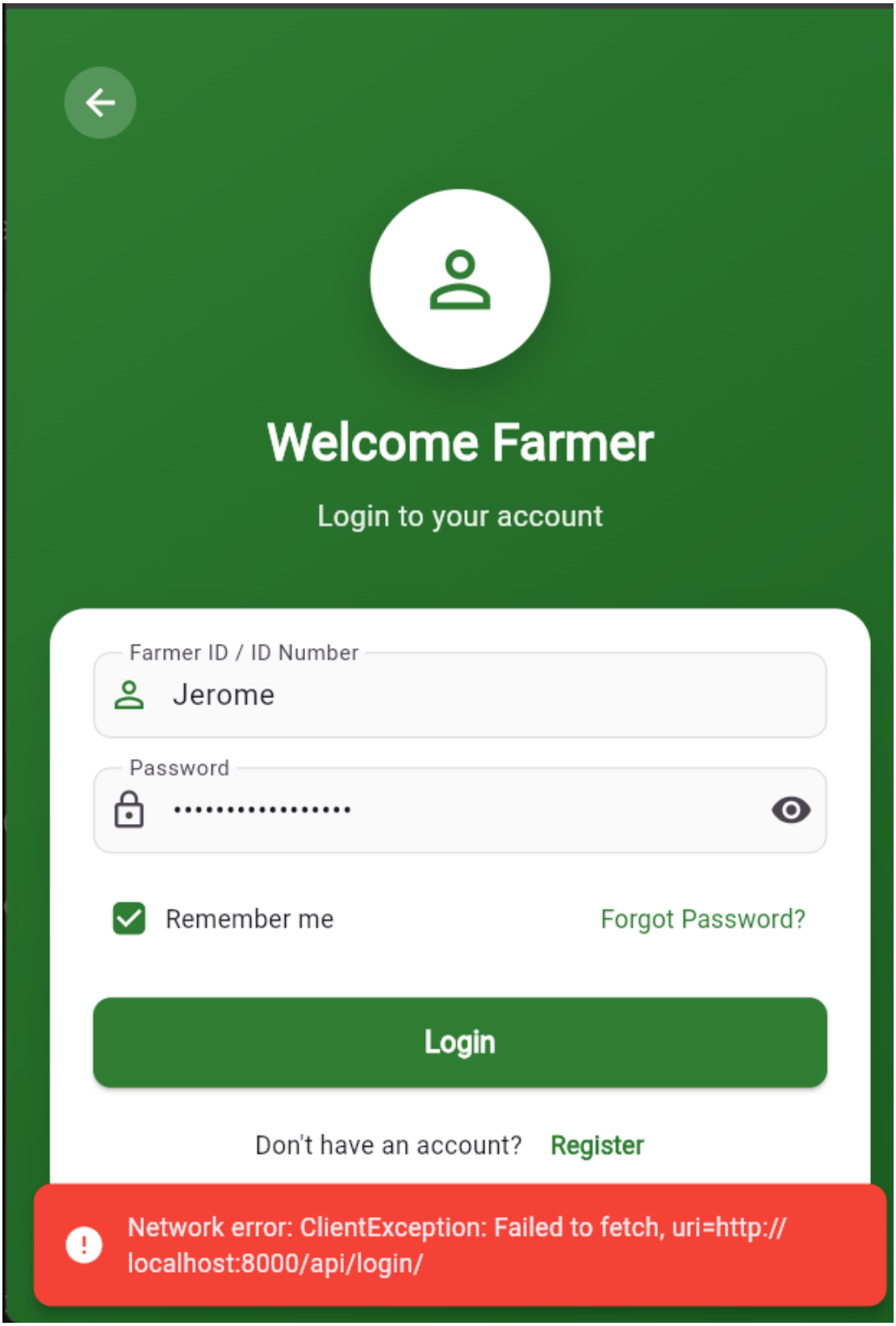

Figure 2: Custom error handling UI

4. FUTURE WIDGET IMPLEMENTATION

4.1 What is a Future?

A Future in Dart represents a potential value or error that will be available at some time in the future. It's used for asynchronous operations like:

- API calls to remote servers
- Reading files from disk
- Database queries
- Any operation that takes time to complete

4.2 Future States

State	Description	UI Representation
Uncompleted	Operation in progress	Loading spinner
Completed with data	Success	Display actual content
Completed with error	Failure	Error message with retry

4.3 UI Components That Depend on Futures

Loading State:

4.4 Real Example from Goat Health Hub

Consultations List Screen:

1. **Initial Load:** Shows a spinning progress indicator with "Loading consultations..."
2. **Data Arrives:** Displays a list of consultation cards with farmer names, messages, and status
3. **Empty Result:** Shows "No consultations yet" with a "Create New" button
4. **Error Occurrence:** Displays error message with retry option

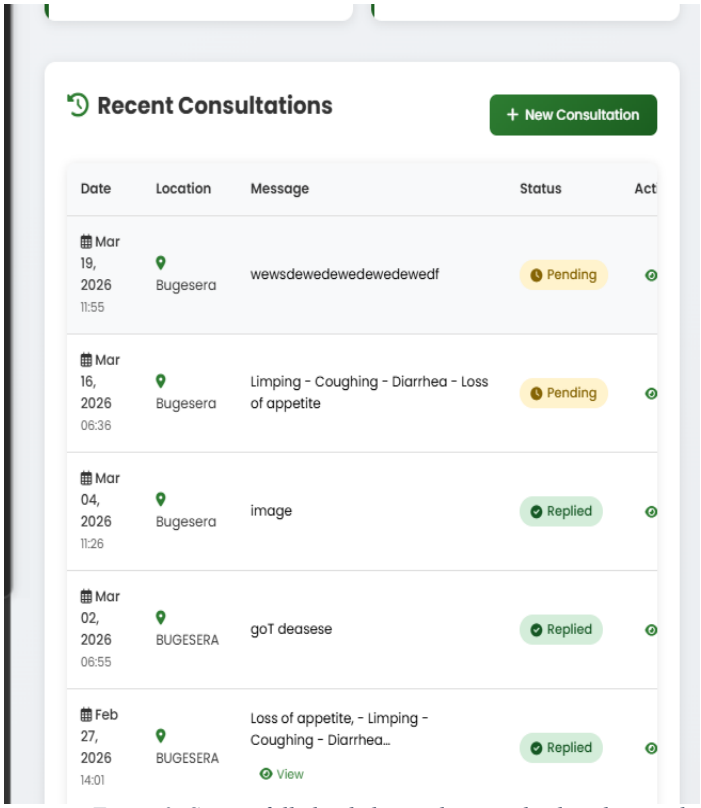

Figure 3: Successfully loaded consultations displayed as cards

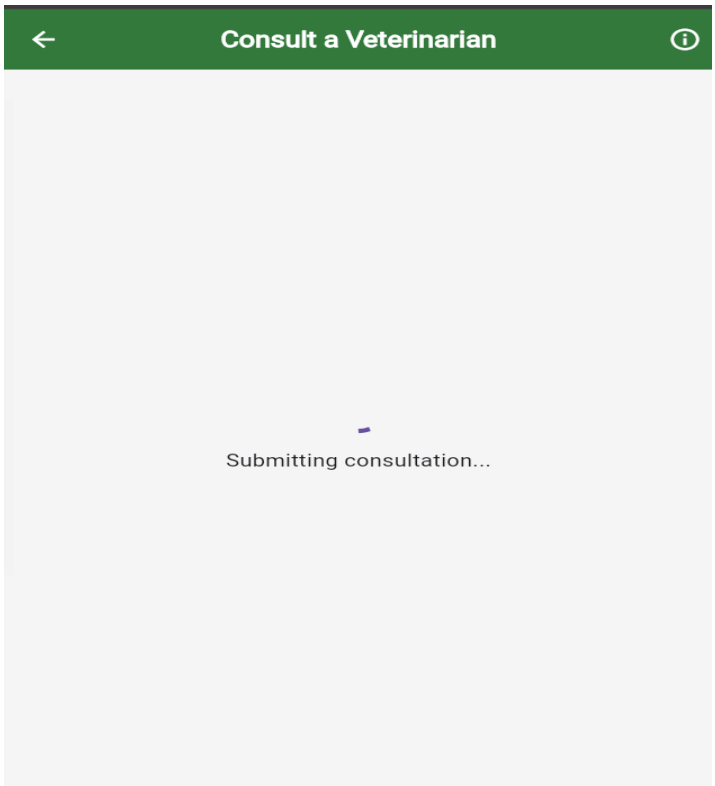

Figure 4:Loading indicator while waiting for API response

5. APPLICATION SCREENSHOTS

5.1 Authentication Screens

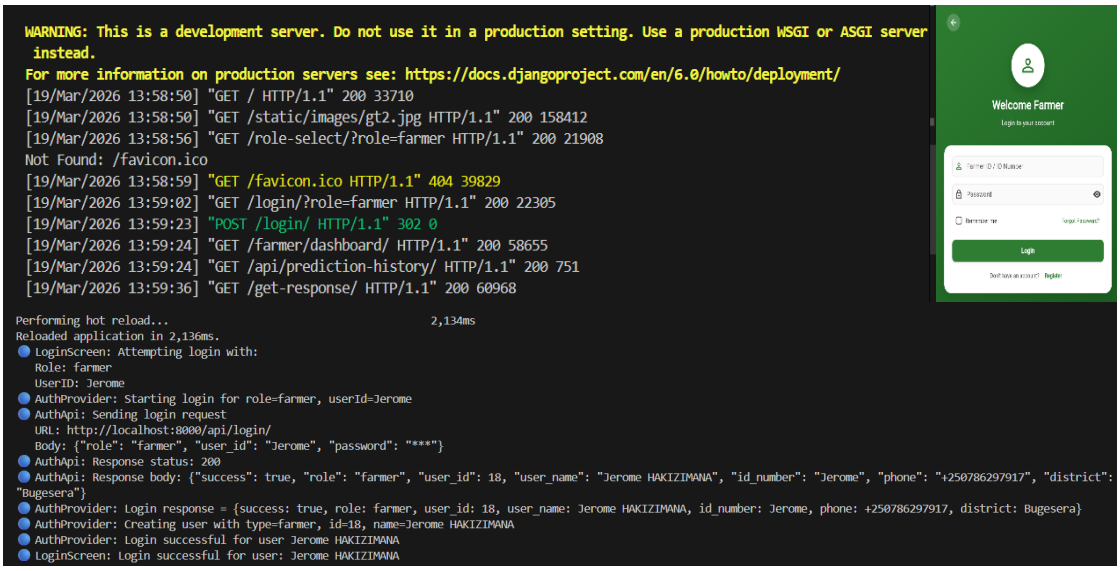

Figure 5:Login screen where users enter credentials

API Called: POST /api/login/
Description: Users enter their role (farmer/vet/admin), ID, and password. The app sends these credentials to the server and receives user data upon successful authentication.

[SCREENSHOT 6: Registration Screen]

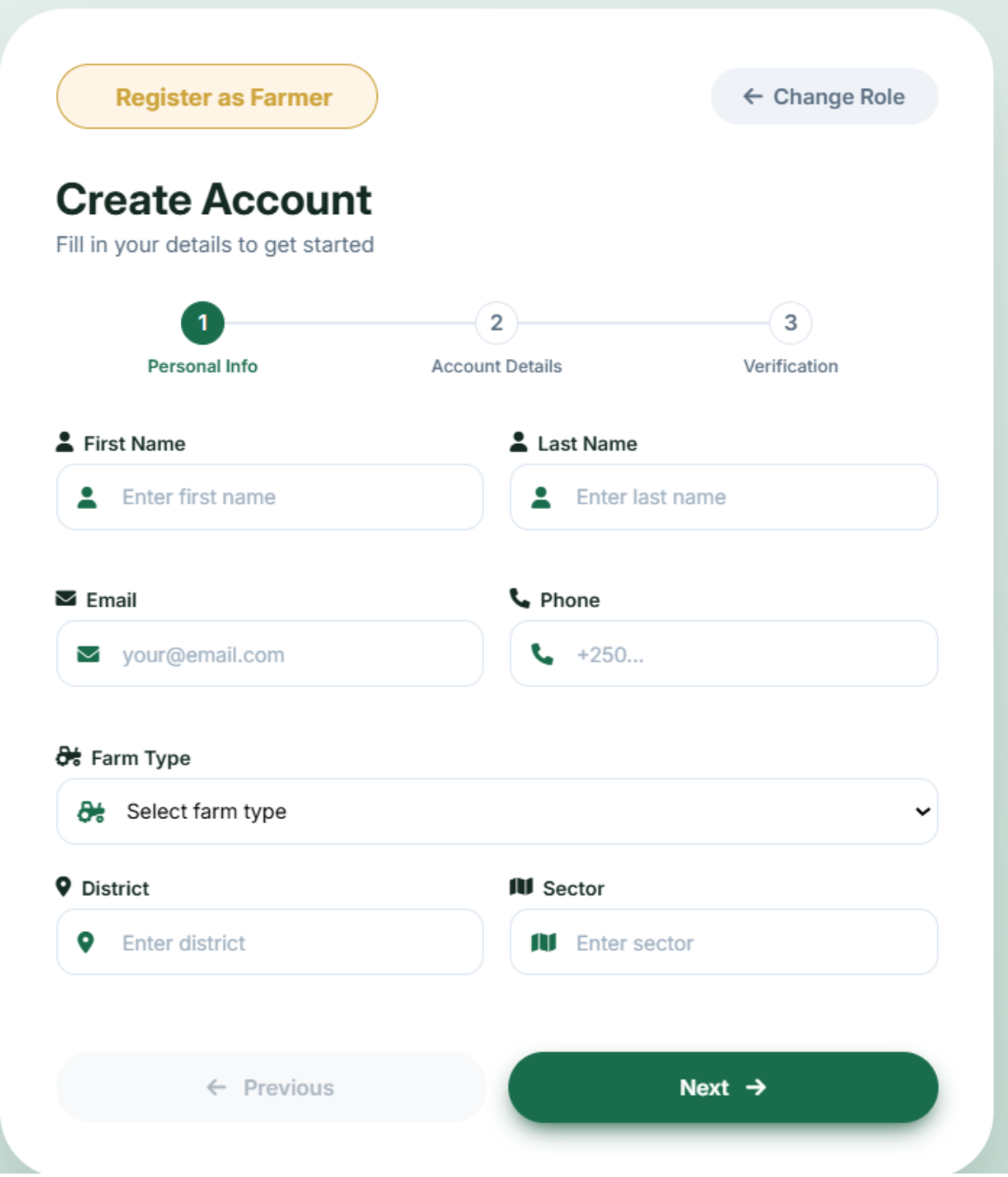

Figure 6: Registration screen for new users

API Called: POST /api/register/
Description: New users fill in their personal information. The app sends this data to create a new account in the system

.5.2 Farmer Dashboard

[SCREENSHOT 7: Farmer Dashboard]

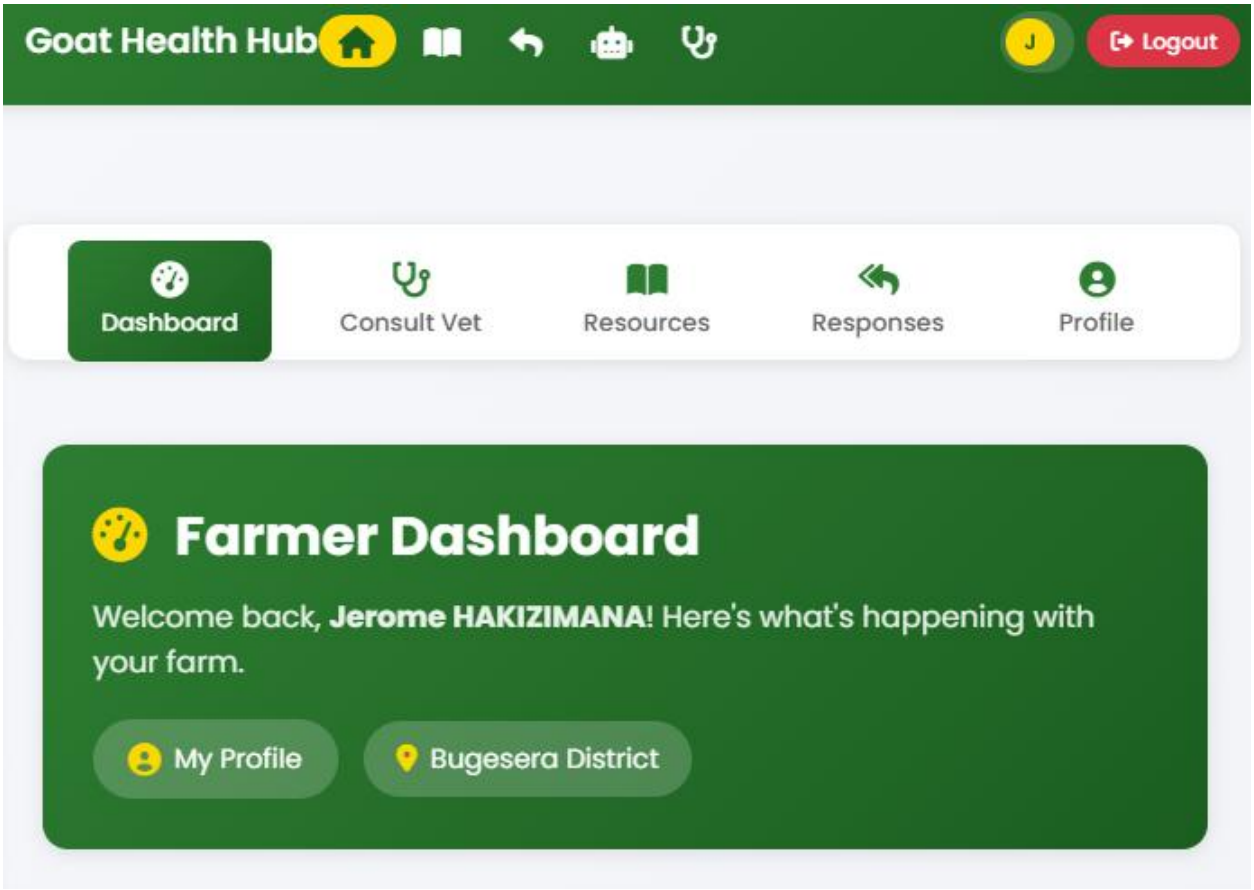

Figure 7: Farmer dashboard showing statistics and quick actions

API Called: GET /api/farmers/[id]/stats/
Description: Displays consultation statistics including total consultations, pending, in progress, and replied counts.

5.3 Consultations

[SCREENSHOT 9: Create Consultation Form]

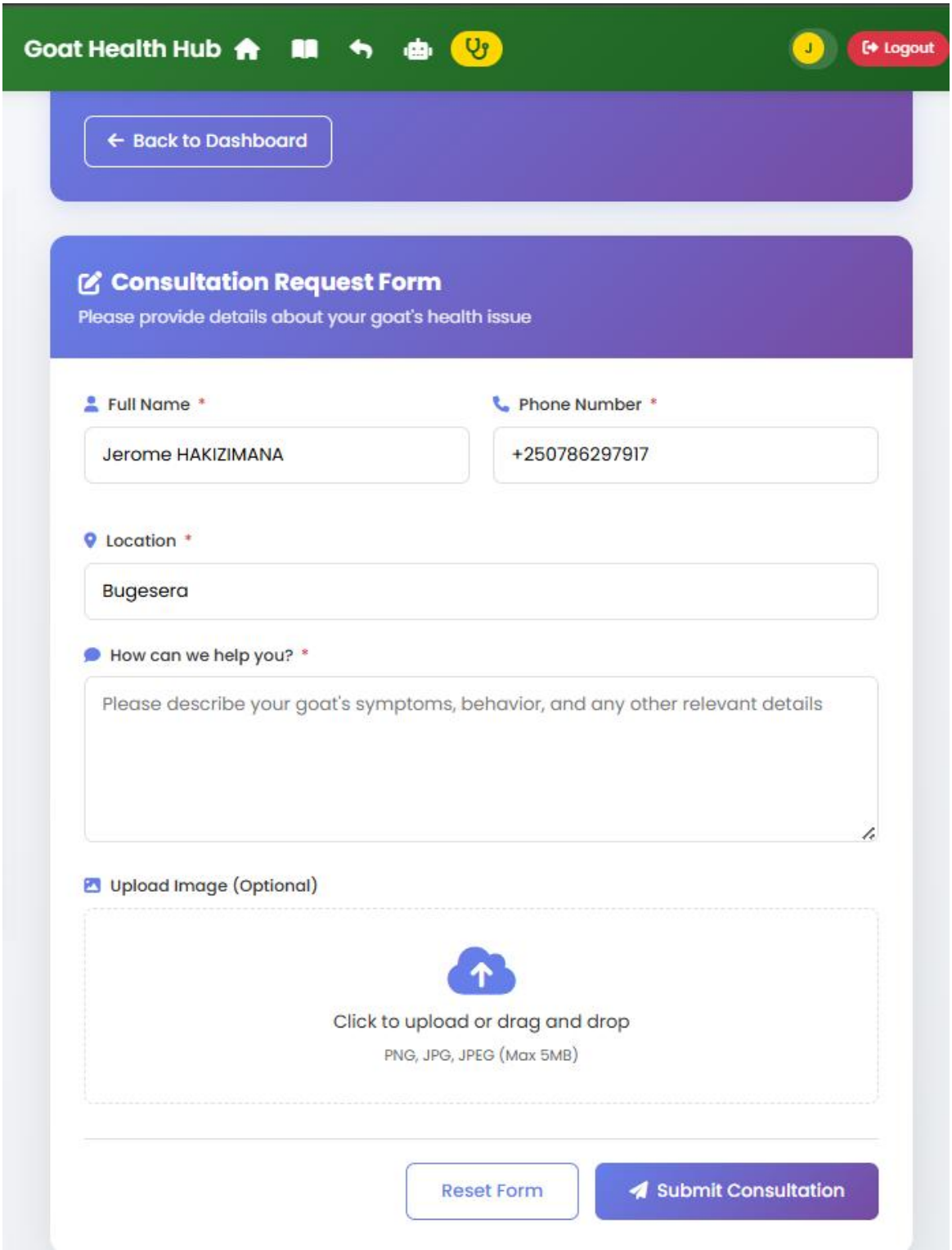

Figure 8: Form for farmers to create a new consultation

API Called: POST /api/consultations/create/
Description: Farmers describe their goat's health issue and optionally attach images. This data is sent to the server for veterinarians to review.

[SCREENSHOT 9: Consultations List (Farmer View)]

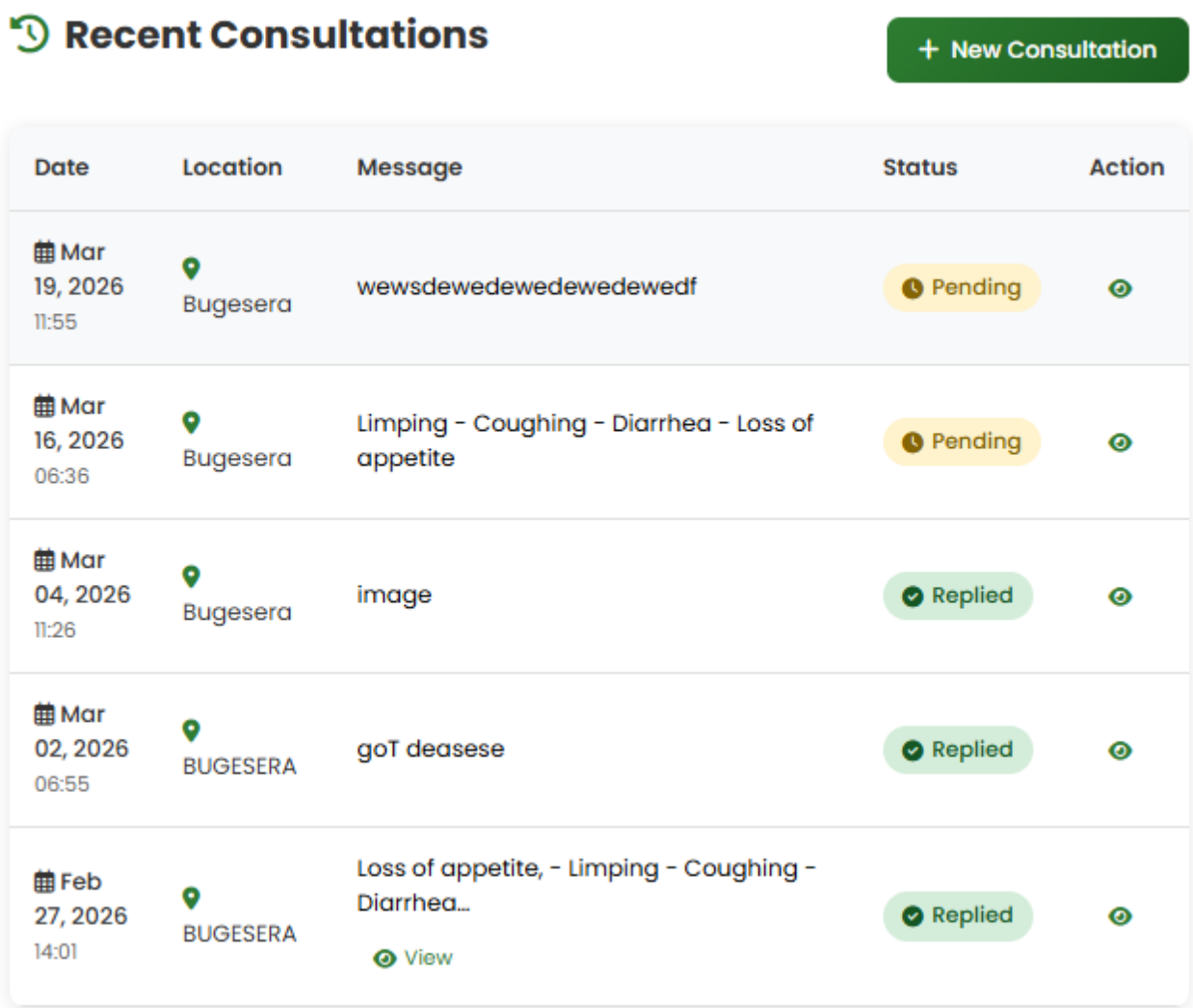

Figure 9: Farmer viewing their consultations with status indicators

API Called: GET /api/farmers/[name]/consultations/
Description: Shows all consultations created by the farmer with color-coded status (pending = orange, in progress = blue, replied = green).

[SCREENSHOT 10: Consultations List (Vet View)]

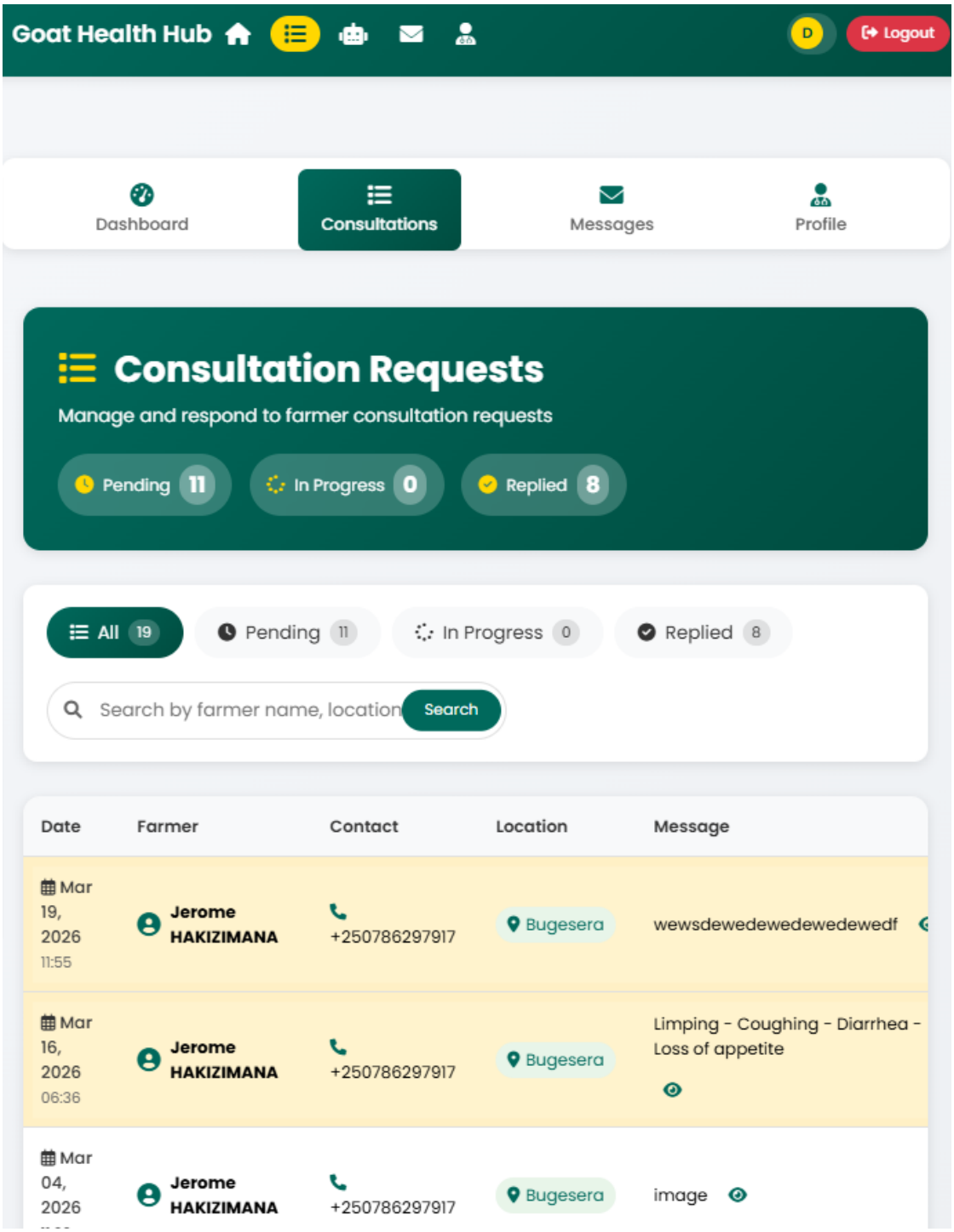

Figure 10: Veterinarian viewing all pending consultations

API Called: GET /api/consultations/
Description: Vets see all consultations with pending ones highlighted, allowing them to prioritize responses.

5.4 Admin Dashboard

[SCREENSHOT 14: Admin Dashboard]

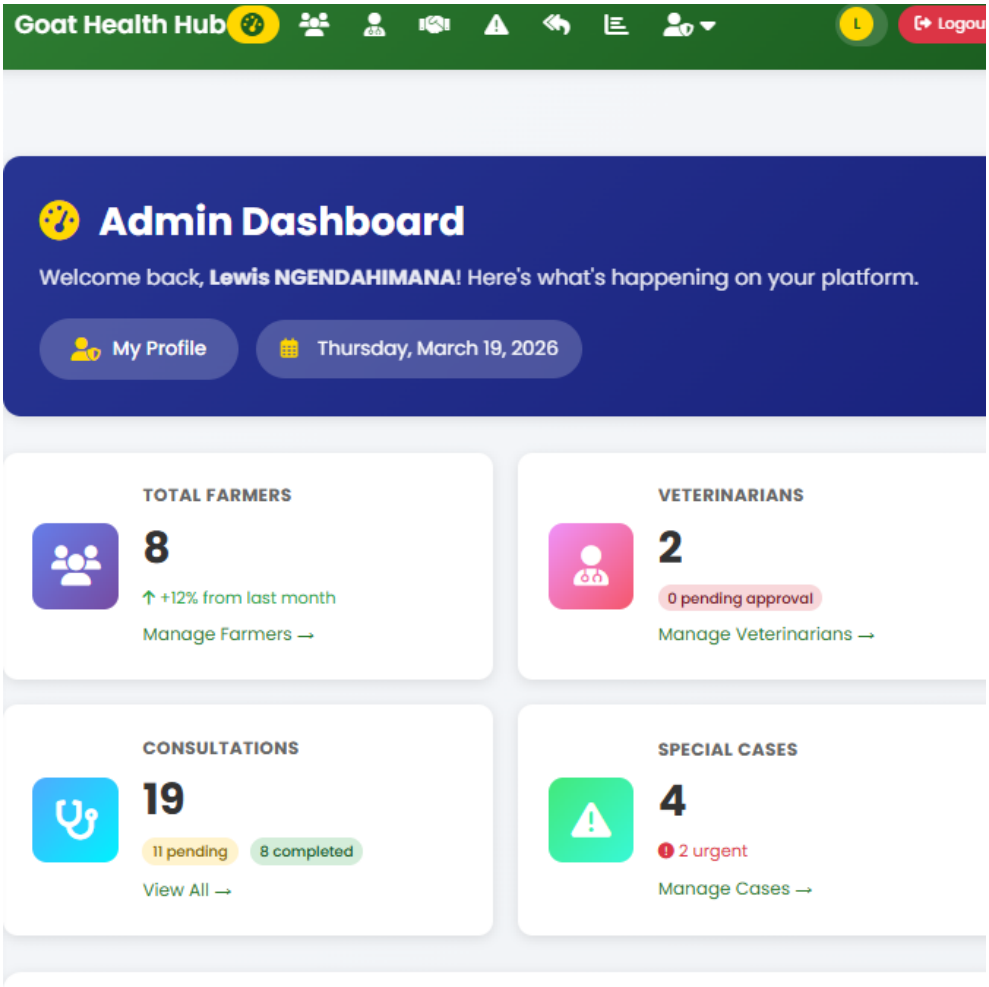

Figure 11: Admin view with system-wide statistics

```
LoginScreen: Attempting login with:
  Role: admin
  UserID: ADMLENG6A7803
AuthProvider: Starting login for role=admin, userId=ADMLENG6A7803
AuthApi: Sending login request
  URL: http://localhost:8000/api/login/
  Body: {"role": "admin", "user_id": "ADMLENG6A7803", "password": "***"}
AuthApi: Response status: 200
AuthApi: Response body: {"success": true, "role": "admin", "user_id": "ADMLENG6A7803", "user_name": "Lewis NGENDAHIMANA", "email": "lewisnge7@gmail.com", "employee_id":
"223026674", "department": "operations"}
AuthProvider: Login response = {success: true, role: admin, user_id: ADMLENG6A7803, user_name: Lewis NGENDAHIMANA, email: lewisnge7@gmail.com, employee_id: 223026674,
department: operations}
AuthProvider: Creating user with type=admin, id=ADMLENG6A7803, name=Lewis NGENDAHIMANA
AuthProvider: Login successful for user Lewis NGENDAHIMANA
LoginScreen: Login successful for user: Lewis NGENDAHIMANA
```

API Called: GET /api/statistics/

Description: Admins see overview of all platform activity including total farmers, consultations, and pending items.

6. CONCLUSION

In this lab, I successfully demonstrated:

Concept	Implementation in Goat Health Hub
API Integration	All CRUD operations using HTTP package
Error Handling	Custom error widget with retry mechanism
Future Widget	Loading states for all async operations
State Management	Provider pattern for reactive UI
Authentication	JWT token storage using SharedPreferences
File Upload	Image picker for consultation attachments
Navigation	Role-based routing with Go Router

Key Learning Outcomes:

1. **HTTP Requests:** Mastered GET, POST, PUT, DELETE operations with proper headers and body formatting
2. **Async Programming:** Understood Futures, async/await, and how to manage UI states during async operations
3. **Error Resilience:** Built robust error handling that provides good user experience even when things go wrong
4. **Clean Architecture:** Separated concerns into models, providers, services, and UI layers
5. **User Experience:** Implemented loading indicators, empty states, and error screens for all async operations

The Goat Health Hub app successfully demonstrates all required API integration concepts while providing a practical solution for real users.

APPENDIX: SCREENSHOT INDEX

Figure	Screen Name	API Endpoint
1	pubspec.yaml	N/A
2	Error Widget	N/A
3	Loading State	N/A
4	Consultations List	GET /api/consultations/
5	Empty State	N/A
6	Login Screen	POST /api/login/
7	Registration	POST /api/register/
8	Farmer Dashboard	GET /api/farmers/[id]/stats/
9	Create Consultation	POST /api/consultations/create/
10	Farmer Consultations	GET /api/farmers/[name]/consultations/
11	Vet Consultations	GET /api/consultations/
12	Reply Form	POST /api/replies/create/
13	Responses	GET /api/consultations/[id]/replies/
14	Admin Dashboard	GET /api/statistics/
15	Profile	GET /api/farmers/[id]/

----------END----------